WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Course Review

Lecture 23: from switches to a computer, and the exam

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 The whole journey

02 Review by learning outcome

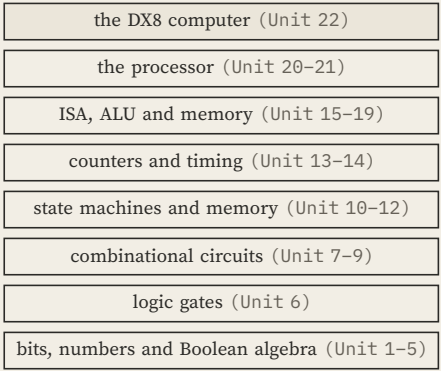
03 The exam

04 Where next

01

The whole journey

From switches to a computer



- Each layer is built only from the one below it.
- The first half of the module: the foundations of digital design. The second half: putting them together into a working 8-bit computer.
- The six learning outcomes of the module follow the same order: the next slides take them one at a time.

02

Review by learning outcome

LO1: numbers

You should be able to

- tell additive from positional numeral systems;
- convert between decimal, binary, octal and hexadecimal;
- group binary digits in threes (octal) or fours (hexadecimal).

Units 1 and 2

Practice

1. Write `0x2F` in binary and in decimal.
2. Write 156 in octal.

LO1: numbers

You should be able to

- tell additive from positional numeral systems;
- convert between decimal, binary, octal and hexadecimal;
- group binary digits in threes (octal) or fours (hexadecimal).

Units 1 and 2

Practice

1. Write `0x2F` in binary and in decimal.
   0010 1111 = 47.
2. Write 156 in octal.

LO1: numbers

You should be able to

- tell additive from positional numeral systems;
- convert between decimal, binary, octal and hexadecimal;
- group binary digits in threes (octal) or fours (hexadecimal).

Units 1 and 2

Practice

1. Write `0x2F` in binary and in decimal.
   0010 1111 = 47.
2. Write 156 in octal.
   $156 = 2 \cdot 64 + 3 \cdot 8 + 4$: 234_8.

LO2: Boolean algebra

You should be able to

- use the laws and theorems of Boolean algebra, De Morgan's among them;
- write the canonical sum of products and product of sums of a truth table;
- simplify a function of up to four variables with a Karnaugh map.

Units 3, 4 and 5

Practice

1. Simplify $xy + xy' + x'y$.
2. Simplify $f(x, y, z) = \sum m(1, 3, 5, 6, 7)$ with a K-map.

LO2: Boolean algebra

You should be able to

- use the laws and theorems of Boolean algebra, De Morgan's among them;
- write the canonical sum of products and product of sums of a truth table;
- simplify a function of up to four variables with a Karnaugh map.

Units 3, 4 and 5

Practice

1. Simplify $xy + xy' + x'y$.
   $x(y + y') + x'y = x + x'y = x + y$.
2. Simplify $f(x, y, z) = \sum m(1, 3, 5, 6, 7)$ with a K-map.

LO2: Boolean algebra

You should be able to

- use the laws and theorems of Boolean algebra, De Morgan's among them;
- write the canonical sum of products and product of sums of a truth table;
- simplify a function of up to four variables with a Karnaugh map.

Units 3, 4 and 5

Practice

1. Simplify $xy + xy' + x'y$.
   $x(y + y') + x'y = x + x'y = x + y$.
2. Simplify $f(x, y, z) = \sum m(1, 3, 5, 6, 7)$ with a K-map.
   $f = z + xy$.

LO3: combinational circuits

You should be able to

- build any function from a universal gate set, NAND alone included;
- design multiplexers, decoders, encoders, adders and comparators;
- find propagation and contamination delays, and spot glitches.

Units 6 to 9, 16 and 17

Practice

1. Build $x + y$ from NAND gates only.
2. How many select bits does a 32-to-1 multiplexer need?
3. What is -100 as an 8-bit two's complement number?

LO3: combinational circuits

You should be able to

- build any function from a universal gate set, NAND alone included;
- design multiplexers, decoders, encoders, adders and comparators;
- find propagation and contamination delays, and spot glitches.

Units 6 to 9, 16 and 17

Practice

1. Build $x + y$ from NAND gates only.
   $x + y = (x' \, y')'$: NAND the outputs of $\text{NAND}(x, x)$ and $\text{NAND}(y, y)$.
2. How many select bits does a 32-to-1 multiplexer need?
3. What is -100 as an 8-bit two's complement number?

LO3: combinational circuits

You should be able to

- build any function from a universal gate set, NAND alone included;
- design multiplexers, decoders, encoders, adders and comparators;
- find propagation and contamination delays, and spot glitches.

Units 6 to 9, 16 and 17

Practice

1. Build $x + y$ from NAND gates only.
   $x + y = (x' \, y')'$: NAND the outputs of $\mathrm{NAND}(x, x)$ and $\mathrm{NAND}(y, y)$.
2. How many select bits does a 32-to-1 multiplexer need?
   5, since $2^5 = 32$.
3. What is -100 as an 8-bit two's complement number?

LO3: combinational circuits

You should be able to

- build any function from a universal gate set, NAND alone included;
- design multiplexers, decoders, encoders, adders and comparators;
- find propagation and contamination delays, and spot glitches.

Units 6 to 9, 16 and 17

Practice

1. Build $x + y$ from NAND gates only.
   $x + y = (x' \, y')'$: NAND the outputs of $\mathrm{NAND}(x, x)$ and $\mathrm{NAND}(y, y)$.
2. How many select bits does a 32-to-1 multiplexer need?
   5, since $2^5 = 32$.
3. What is -100 as an 8-bit two's complement number?
   1001 1100.

LO4: sequential circuits

You should be able to

- model a problem as a Moore or a Mealy machine, and simplify it;
- explain the SR and D latches and the D, T and JK flip-flops;
- build registers and counters;
- apply the setup and hold time constraints.

Units 10 to 14

Practice

1. A 4-bit up counter starts at 0. What does it show after 20 rising edges?

2. $t_{\text{clk2q}} = 50$ ps, $t_{\text{pd}} = 400$ ps, $t_{\text{setup}} = 50$ ps. What is the fastest clock?

LO4: sequential circuits

You should be able to

- model a problem as a Moore or a Mealy machine, and simplify it;
- explain the SR and D latches and the D, T and JK flip-flops;
- build registers and counters;
- apply the setup and hold time constraints.

Units 10 to 14

Practice

1. A 4-bit up counter starts at 0. What does it show after 20 rising edges?
   20 mod 16 = 4: 0100.
2. $t_{\text{clk2q}} = 50$ ps, $t_{\text{pd}} = 400$ ps, $t_{\text{setup}} = 50$ ps. What is the fastest clock?

LO4: sequential circuits

You should be able to

- model a problem as a Moore or a Mealy machine, and simplify it;
- explain the SR and D latches and the D, T and JK flip-flops;
- build registers and counters;
- apply the setup and hold time constraints.

Units 10 to 14

Practice

1. A 4-bit up counter starts at 0. What does it show after 20 rising edges?
   $20 \bmod 16 = 4$: 0100.
2. $t_{\text{clk2q}} = 50$ ps, $t_{\text{pd}} = 400$ ps, $t_{\text{setup}} = 50$ ps. What is the fastest clock?
   $t_{\text{clk}} \geq 500$ ps: 2 GHz.

LO5: architecture, ALU and memory

You should be able to

- explain the Von Neumann architecture and what an ISA is;
- encode and decode DX8 instructions;
- design an ALU, a register file and byte-addressable RAM from basic parts.

Units 15 to 19

Practice

1. Write $M[\texttt{0x33}] \leftarrow$ R6 (a `store`) in hexadecimal.
2. How many address bits does a byte-addressable RAM of 64 bytes need?

LO5: architecture, ALU and memory

You should be able to

- explain the Von Neumann architecture and what an ISA is;
- encode and decode DX8 instructions;
- design an ALU, a register file and byte-addressable RAM from basic parts.

Units 15 to 19

Practice

1. Write $M[\texttt{0x33}] \leftarrow$ R6 (a `store`) in hexadecimal.
   0 0001 110, 0011 0011: `0x0E33`.
2. How many address bits does a byte-addressable RAM of 64 bytes need?

LO5: architecture, ALU and memory

You should be able to

- explain the Von Neumann architecture and what an ISA is;
- encode and decode DX8 instructions;
- design an ALU, a register file and byte-addressable RAM from basic parts.

Units 15 to 19

Practice

1. Write $M[\texttt{0x33}] \leftarrow$ R6 (a `store`) in hexadecimal.
   0 0001 110, 0011 0011: `0x0E33`.
2. How many address bits does a byte-addressable RAM of 64 bytes need?
   6.

LO6: the processor

You should be able to

- explain fetch, decode, execute and load;
- compare single-cycle, multi-cycle and pipelined designs;
- build the DX8 in a circuit simulator, and program it.

Units 20 to 22

Practice

1. 4 stages of 2 cycles each. How many cycles do 20 instructions take, multi-cycle and pipelined?

2. In which step does the DX8 copy the result of an `add` into a register?

LO6: the processor

You should be able to

- explain fetch, decode, execute and load;
- compare single-cycle, multi-cycle and pipelined designs;
- build the DX8 in a circuit simulator, and program it.

Units 20 to 22

Practice

1. 4 stages of 2 cycles each. How many cycles do 20 instructions take, multi-cycle and pipelined? 160, and $2 \cdot (20 + 3) = 46$.
2. In which step does the DX8 copy the result of an `add` into a register?

LO6: the processor

You should be able to

- explain fetch, decode, execute and load;
- compare single-cycle, multi-cycle and pipelined designs;
- build the DX8 in a circuit simulator, and program it.

Units 20 to 22

Practice

1. 4 stages of 2 cycles each. How many cycles do 20 instructions take, multi-cycle and pipelined?
   160, and $2 \cdot (20 + 3) = 46$.
2. In which step does the DX8 copy the result of an `add` into a register?
   s_7, in the load stage.

03

The exam

How the module is assessed

Component	Weight	Due	Outcomes
Quizzes	20%	throughout	LO1 to LO5
Circuit-design project: the DX8 computer	40%	throughout	LO3 to LO6
Final exam	40%	end of module	LO1 to LO6

- The final exam covers every learning outcome: every unit of the module.
- Any support materials that are permitted will be provided during the exam. No other aids or resources are allowed.

The final exam

- Date and time: `TODO(owner): exam date and time`
- Length: `TODO(owner): length of the exam`
- Format and kinds of questions: `TODO(owner): exam format`
- Support materials provided: `TODO(owner): which materials are provided`

How to prepare

Work through the exercise sheets and the “Try it” slides of every lecture again, rebuild the key circuits in Logisim-evolution, and practise encoding and tracing DX8 programs by hand.

04

Where next

Where next

- You have built a computer from switches that are on or off, one layer of abstraction at a time.
- *Digital Design and Computer Architecture II* (CS221-F27, Semester 3) continues with real instruction set architectures, memory systems, input and output, and processor performance.
- Keep your DX8: extend it, program it, and check its timing.

> **Remark**
>
> The book's epilogue says that it kept many explanations short, even hand-wavy, to stay under 100 pages, and it asks sharp readers for feedback. Finding those gaps yourself is the next step in learning.